16-bit Adder Architecture Comparison: Ripple Carry and Carry Lookahead

Hamza Ikram
Higher National School of Nanoscience and Nanotechnology
Algiers, Algeria
i.hamza@nsnn.edu.dz

***Abstract*—This project compares two 16-bit adder architectures: the Ripple Carry Adder (RCA) and the Carry Lookahead Adder (CLA). Both architectures were implemented using the Verilog hardware description language and verified through simulation. Their timing behavior was analyzed using a simplified unit-delay model. The results show that the CLA is faster than the RCA, with a shorter critical path resulting from the parallel generation of carry signals, compared with the sequential carry propagation in the RCA.**

Index Terms—16-bit adder, Ripple Carry Adder, Carry Lookahead Adder, Verilog, timing analysis, unit-delay model

I. Introduction

Adders are fundamental building blocks in arithmetic logic units (ALUs) and digital systems. Their performance can directly affect the speed of arithmetic operations; therefore, improving the speed of an adder can contribute to faster arithmetic computation overall. In this work, two 16-bit adder architectures are considered: the Ripple Carry Adder (RCA) and the Carry Lookahead Adder (CLA). The two architectures are compared under the same timing assumptions in order to study their propagation behavior. A simplified unit-delay model is used to make the timing difference between the two architectures easier to analyze. By applying this model and analyzing signal arrival times, the critical paths and resulting delays of the two architectures can be determined.

The results show that the CLA achieves a shorter delay than the RCA under the adopted model. This improvement comes at the cost of additional hardware complexity, since the CLA uses extra logic to generate carry signals without waiting for the carry to propagate sequentially through all preceding stages. In contrast, the RCA relies on sequential carry propagation from one stage to the next.

II. RCA and CLA Architectures

A. Full Adder

$$S = A \oplus B \oplus C_{in} \tag{1}$$

$$C_{out} = AB + C_{in}(A \oplus B) \tag{2}$$

A full adder is a basic combinational circuit used to perform binary addition. It adds three one-bit inputs, A, B, and Cin, and produces two outputs: the sum S and the carry-out Cout. The output carry is then used as the carry input of the next stage when multiple full adders are cascaded to form a multi-bit adder.

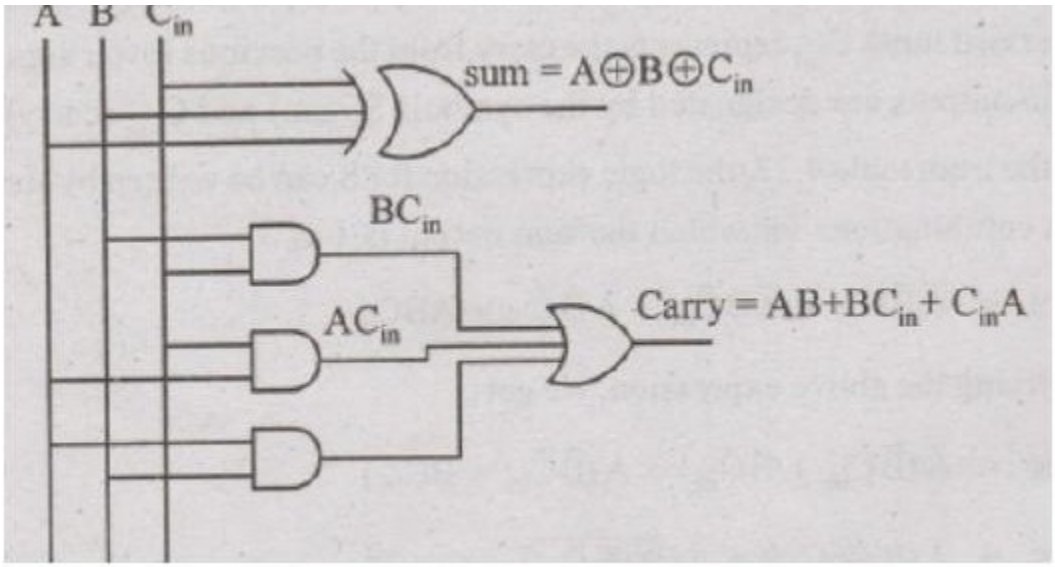

Fig. 1. Full adder gate-level implementation.

B. Ripple Carry Adder

A Ripple Carry Adder (RCA) is constructed by cascading multiple full adders. For n-bit addition, n full adders are connected such that the carry output of each stage is connected to the carry input of the next stage. In the 16-bit RCA, the first full adder receives the external carry input, while each subsequent stage receives the carry generated by the previous stage. Since each stage depends on the carry generated by the preceding stage, the carry propagates sequentially from the least significant bit to the most significant bit. This sequential dependency creates a potentially long critical path, since the final carry cannot be determined until the preceding carry signals have propagated through the chain. The resulting timing behavior is analyzed later using the unit-delay model.

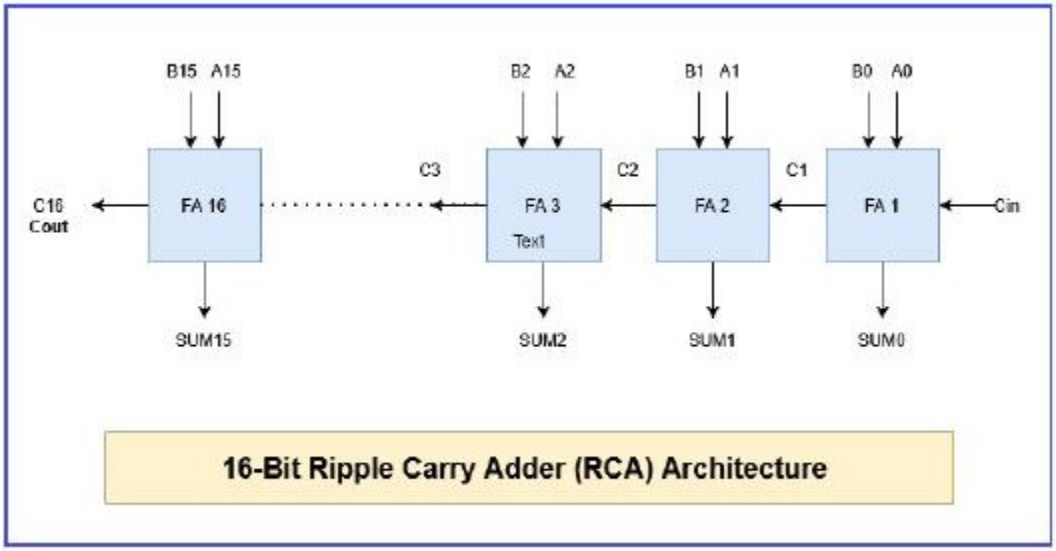

Fig. 2. 16-bit Ripple Carry Adder.

C. Carry Lookahead Adder

A Carry Lookahead Adder (CLA) is designed to reduce the delay caused by sequential carry propagation in a Ripple Carry Adder. Instead of waiting for the carry to propagate from one stage to the next, the CLA uses generate and propagate signals to determine carry signals in advance.

$$P_i = A_i \oplus B_i \tag{3}$$

$$G_i = A_i B_i \tag{4}$$

$$C_{i+1} = G_i + P_i C_i \tag{5}$$

Why is the CLA faster?

$$C_1 = G_0 + P_0 C_0 \tag{6}$$

$$C_2 = G_1 + P_1 G_0 + P_1 P_0 C_0 \tag{7}$$

$$C_3 = G_2 + P_2 G_1 + P_2 P_1 G_0 + P_2 P_1 P_0 C_0 \tag{8}$$

In this project, the 16-bit CLA is organized as four 4-bit CLA blocks. A lookahead carry unit generates the carries between these blocks using their group propagate and group generate signals. The RCA and CLA therefore differ mainly in their carry computation strategy: the RCA relies on sequential carry propagation, whereas the CLA uses additional logic to compute carry signals in advance. The timing implications of these two approaches are analyzed using the unit-delay model in the following section.

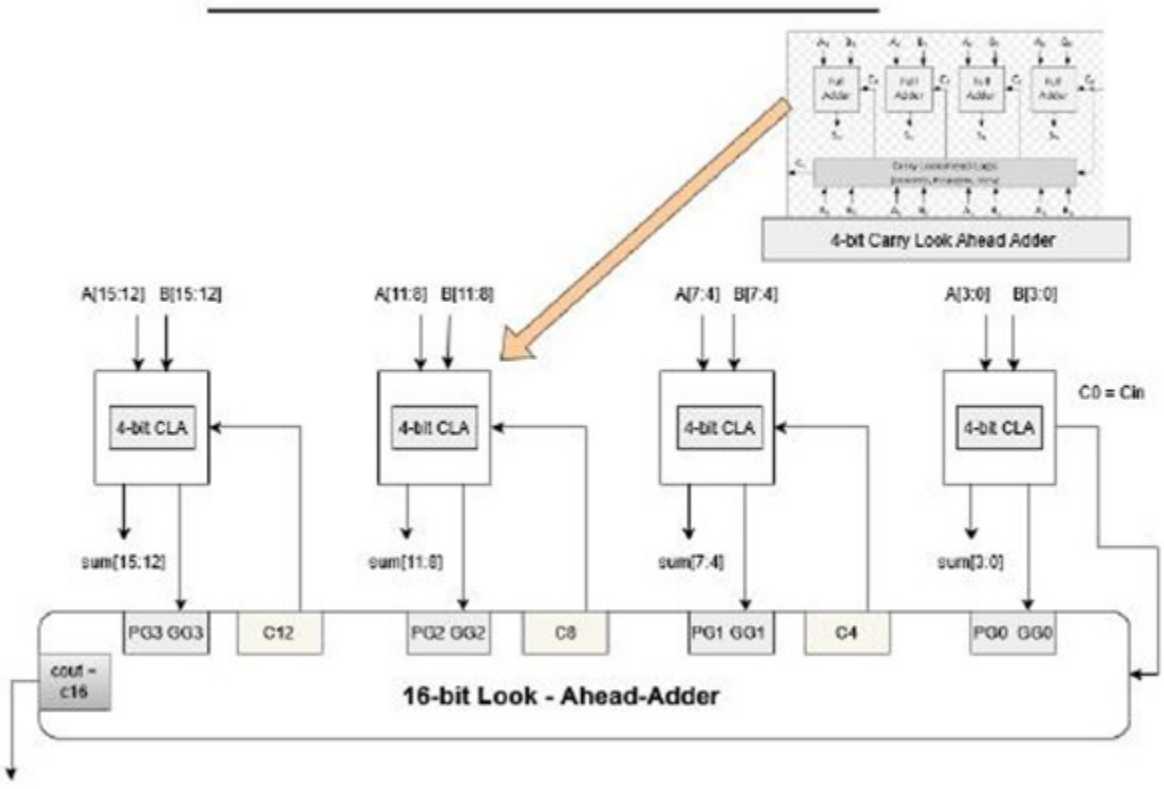

Fig. 3. 16-bit Carry Lookahead Adder.

III. Timing Comparison Under the Unit-Delay Model

A. Unit-Delay Model

To compare the timing behavior of the RCA and CLA architectures under the same conditions, a simplified unit-delay model is used. In this model, each logic operation is assumed to introduce one unit of delay, denoted by τ. This model does not represent the physical delay of a specific CMOS technology; instead, it provides a simplified way to compare the logical depth and critical paths of the two architectures.

B. Arrival-Time Analysis

The arrival time of a signal represents the earliest time at which the signal becomes stable and correct. Primary inputs are assumed to be available at time zero. For a logic gate, the output arrival time is calculated as

$$T_{out} = \tau + max(T_{in}) \quad (9)$$

where τ is the assumed gate delay. The maximum input arrival time is used because the output cannot become valid until all required inputs have arrived. The longest path through the circuit therefore determines the critical path and the overall propagation delay.

C. RCA Timing Analysis

For the 16-bit Ripple Carry Adder, the carry propagates sequentially from one full adder to the next. Using the full adder carry equation and the unit-delay model, the arrival times of the first carry signals are

$$T_{C1} = 3\tau \quad (10)$$

$$T_{C2} = 5\tau \quad (11)$$

$$T_{C3} = 7\tau \quad (12)$$

The resulting pattern is

$$T_{Cn} = (2n + 1)\tau \quad (13)$$

Therefore, for the 16-bit RCA:

$$T_{C16} = 33\tau \quad (14)$$

The carry chain forms the critical path because each stage depends on the carry generated by the previous stage.

D. CLA Timing Analysis

The Carry Lookahead Adder reduces sequential carry propagation by using propagate and generate signals to calculate carries ahead of time. For each 4-bit block, the group propagate and group generate signals are defined as

$$PG = P_3P_2P_1P_0 \quad (15)$$

$$GG = G_3 + P_3G_2 + P_3P_2G_1 + P_3P_2P_1G_0 \quad (16)$$

The arrival time of the group propagate signal is 2τ, while the group generate signal arrives at 3τ. The lookahead carry unit (LCU) uses these group signals to generate the block-boundary carries directly, without waiting for the carry to ripple through every preceding bit:

$$C_4 = GG_0 + PG_0C_0 \tag{18}$$

$$C_8 = GG_1 + PG_1GG_0 + PG_1PG_0C_0 \tag{19}$$

$$C_{12} = GG_2 + PG_2GG_1 + PG_2PG_1GG_0 + PG_2PG_1PG_0C_0 \tag{20}$$

$$C_{16} = GG_3 + PG_3GG_2 + PG_3PG_2GG_1 + PG_3PG_2PG_1GG_0 + PG_3PG_2PG_1PG_0C_0 \tag{21}$$

Evaluating these expressions under the unit-delay model gives the block-boundary carry arrival times summarized in Table I. Each block carry is obtained directly from the group signals of the current and preceding blocks, so its arrival time no longer grows linearly with bit position the way it does in the RCA.

TABLE I

ARRIVAL TIME OF BLOCK-BOUNDARY CARRIES (LCU)

Carry	Arrival Time
C4	4τ
C8	5τ
C12	5τ
C16	5τ

E. Local Carry Generation and the Sum Critical Path

Once the LCU supplies the incoming carry for each 4-bit block, the internal carries within that block are generated locally using the standard 4-bit CLA equations, e.g. for a block starting at bit j: Cj+1 = Gj + PjCj, Cj+2 = Gj+1 + Pj+1Gj + Pj+1PjCj, and so on. Because block 0 starts from C0 = 0, its internal carries arrive at 3τ. Blocks 1–3, however, start from the LCU-generated carries C4 = 4τ, C8 = 5τ, and C12 = 5τ, so their internal carries arrive two units later, at 6τ and 7τ respectively — the LCU output does not wait for these internal carries, but the sum outputs of each block do depend on them.

Since Sumi = Pi XOR Ci, the arrival time of each sum output is τ + max(Arrival(Pi), Arrival(Ci)). The latest internal carries (7τ, in blocks 1–3) therefore produce sum outputs arriving at 8τ. This is why the overall critical path of the CLA is not set by the final carry C16, which arrives at only 5τ, but by the sum outputs of the upper bits within each block, which arrive at 8τ:

$$T_{CLA} = 8\tau \tag{22}$$

F. Timing Comparison

The overall timing results obtained from the unit-delay analysis are summarized in Table II.

TABLE II

TIMING COMPARISON UNDER THE UNIT-DELAY MODEL

Architecture	Critical Path	Delay
RCA16	Carry chain	33τ
CLA16	Lookahead logic	8τ

IV. Functional Verification

A. Verilog Implementation

The 16-bit RCA and CLA architectures were implemented using Verilog. The designs were constructed hierarchically from smaller building blocks, including the full adder and the 4-bit CLA blocks. Separate testbenches were used to apply different input combinations and verify the functional behavior of each architecture.

B. Icarus Verilog Simulation

Icarus Verilog was used to compile and simulate the Verilog designs. Several input combinations were applied through dedicated testbenches, including zero values, carry-producing additions, maximum-value additions, and representative hexadecimal operands. The simulations were used to verify that the calculated sum and carry outputs matched the expected binary addition results.

C. GTKWave Results

The simulation output was stored as a VCD waveform and inspected using GTKWave. The waveform provides visual confirmation that the input combinations produce the expected sum and carry outputs.

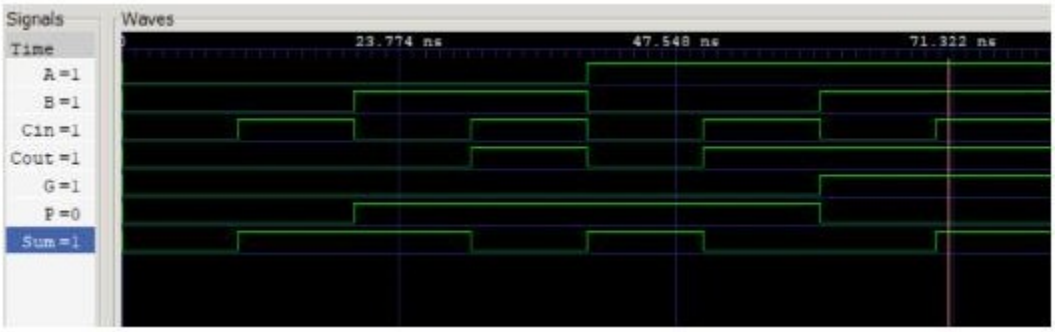

Fig. 4. Full adder waveform.

V. Conclusion

This work presented a comparison between the 16-bit Ripple Carry Adder (RCA) and Carry Lookahead Adder (CLA) architectures. Both designs were implemented in Verilog and functionally verified through Icarus Verilog simulation and GTKWave waveform analysis. A simplified unit-delay model was then used to analyze signal arrival times and compare the critical paths of the two architectures. The analysis resulted in a delay of 33τ for the RCA and 8τ for the CLA. The shorter delay of the CLA is achieved by reducing sequential carry propagation through lookahead carry generation. These results demonstrate the timing advantage of the CLA under the adopted model, while also highlighting the additional logic complexity required by the lookahead architecture.

VI. Future Work

Future work may include more realistic gate-delay modeling, synthesis-based timing and area analysis, power evaluation, and FPGA implementation. These extensions would provide a more realistic comparison of the two architectures beyond the simplified unit-delay model.